



HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Quiz # 2 Solution

QUESTION 1: ALGORITHM DESIGN USING INSERTION SORT (10 marks)

- ICC Champions Cricket Trophy scores:
 - ICC Champions Cricket Trophy 2025 is happening in full swing successfully in Pakistan (well most of it is, at least 😊). Each team's batting lineup performance is being recorded in a grid in such a way that player's name and score is placed in an M-by-N table, where M represents the number of rows, and N represents the number of columns of the table. Each row represents the series of matches they played, and each column representing the team's batting order for that match. Assume this grid is given in correct form, so no need to take input or do error checking, however, do declare/define the grid and its size parameters in your algorithm
 - To analyze the performance of each player in a match, a new scoreboard table is generated of size i-by-j where i represents each player's performance sorted alphabetically, and j represents their performance scores sorted in descending order.
 - This is done by applying the following steps:
 - Sort each row of the grid alphabetically by player name (A to Z) using Insertion Sort
 - Transform the grid into scorecard table
 - Sort each row of the scorecard table in descending order of scores using Insertion Sort
 - Each entry in the grid and scorecard table is a tuple that has the player's name as the first entry of the tuple and the second element representing their score in that match. If a specific player in the team lineup did not participate in the match, their score is defined as -1, in order to distinguish them from the players who did participate and made 0 runs only.
 - For example:
 - Pakistan's team plays 3 matches in the tournament
 - The grid for the team players will be as follows:
 - [[('Saud', 6), ('Babar', 64), ('Rizwan', 3), ('Imam', 24), ('Salman', 42), ('Tayyab', 1), ('Khushdil', 69), ('Shaheen', 14), ('Naseem', 13), ('Haris', 19), ('Abrar', 0)],
[('Imam', 10), ('Babar', 23), ('Saud', 62), ('Rizwan', 46), ('Tayyab', 4), ('Salman', 19), ('Khushdil', 38), ('Shaheen', 0), ('Naseem', 14), ('Haris', 8), ('Abrar', 10)],
[('Babar', 61), ('Rizwan', 43), ('Saud', 55), ('Imam', 14), ('Salman', 39), ('Tayyab', 0), ('Khushdil', -1), ('Shaheen', 7), ('Abrar', -1), ('Naseem', 0), ('Haris', -1)]]
 - Sorting each row alphabetically by their names to get the updated grid as:
 - [[('Abrar', 0), ('Babar', 64), ('Haris', 19), ('Imam', 24), ('Khushdil', 69), ('Naseem', 13), ('Rizwan', 3), ('Salman', 42), ('Saud', 6), ('Shaheen', 14), ('Tayyab', 1)],
[('Abrar', 10), ('Babar', 23), ('Haris', 8), ('Imam', 10), ('Khushdil', 38), ('Naseem', 14), ('Rizwan', 46), ('Salman', 19), ('Saud', 62), ('Shaheen', 0), ('Tayyab', 4)],
[('Abrar', -1), ('Babar', 61), ('Haris', -1), ('Imam', 14), ('Khushdil', -1), ('Naseem', 0), ('Rizwan', 43), ('Salman', 39), ('Saud', 55), ('Shaheen', 7), ('Tayyab', 0)]]
 - Then transforming the grid into scorecard table and get the results sorted in descending order for scores
 - [[('Abrar', 10), ('Abrar', 0), ('Abrar', -1)], [('Babar', 64), ('Babar', 61), ('Babar', 23)],
[('Haris', 19), ('Haris', 8), ('Haris', -1)], [('Imam', 24), ('Imam', 14), ('Imam', 10)],
[('Khushdil', 69), ('Khushdil', 38), ('Khushdil', -1)], [('Naseem', 14), ('Naseem', 13), ('Naseem', 0)],
[('Rizwan', 46), ('Rizwan', 43), ('Rizwan', 3)], [('Salman', 42), ('Salman', 39), ('Salman', 19)],
[('Saud', 62), ('Saud', 55), ('Saud', 6)], [('Shaheen', 14), ('Shaheen', 7), ('Shaheen', 0)],
[('Tayyab', 4), ('Tayyab', 1), ('Tayyab', 0)]]

- **REQUIREMENTS:**

- Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized grids
- Each of these sorting must be done using Insertion Sort ONLY. Using any other sorting algorithm will result in 0 marks for the question, including built-in sort functions, no matter how accurate or optimized your algorithm is.
- **DO NOT JUST SAY “DO INSERTION SORT ...”, ETC. INSTEAD SPECIFY THE ACTUAL LOGIC FOR IT ON HOW IT IS BEING IMPLEMENTED HERE!**
- Each sorting must be done without flattening the table, i.e., without converting it into a one-dimensional array. Instead, the elements must be rearranged while keeping the grid or scoreboard table structure intact.
- Declare all the data structures with their correct sizes, and other variables being used in this algorithm, including any inputs, if any.
- **DO NOT USE ANY OTHER DATA STRUCTURES INCLUDING ListADT/Python lists apart from the ones permitted to use**
- **REMEMBER: Insertion Sort is an in-place algorithm, which means no additional arrays/lists are generated while performing sorting.**

SOLUTION: I have provided the code for the above program for reference for now. May update it in terms of pseudocode later

```
def insertion_sort_column_by_name(grid):
    """ Sorts the columns of the grid based on player names using Insertion Sort """
    M = len(grid)
    N = len(grid[0])

    for row in range(M):
        for col in range(1, N):
            key = grid[row][col]

            # Sort column by player name (A-Z)
            while col > 0 and grid[row][col-1][0] > key[0]:
                grid[row][col] = grid[row][col-1]
                col -= 1
            grid[row][col] = key

def insertion_sort_column_by_score(scoreboard):
    """ Sorts each column based on score in descending order using Insertion Sort """
    M = len(scoreboard)
    N = len(scoreboard[0])

    for row in range(M):
        for col in range(1, N):
            key = scoreboard[row][col]

            # Sort columns by scores in descending order
            while col > 0 and scoreboard[row][col-1][1] < key[1]:
                scoreboard[row][col] = scoreboard[row][col-1]
                col -= 1
            scoreboard[row][col] = key

def get_scoreboard(grid):
    M = len(grid)
    N = len(grid[0])

    scoreboard = [None] * N
    for i in range(N):
```

```

        scoreboard[i] = [None] * M

    for i in range(M):
        for j in range(N):
            scoreboard[j][i] = grid[i][j]

    return scoreboard

def transform_grid_in_place(grid):
    """ Sorts the grid in-place without flattening it """
    insertion_sort_column_by_name(grid) # Step 1: Sort columns by name
    for row in grid:
        print(row)
    scoreboard = get_scoreboard(grid)
    insertion_sort_column_by_score(scoreboard) # Step 2: Sort each column by score in
descending order
    return scoreboard

# Sample 2D Grid
# grid = [
#     [("Haris", 25), ("Shoaib", 100), ("Babar", 53), ("Faheem", 98), ("Miandad", 110)],
#     [("Haris", 125), ("Shoaib", 10), ("Babar", -1), ("Faheem", 12), ("Miandad", 100)],
#     [("Haris", 33), ("Shoaib", 45), ("Babar", 0), ("Faheem", 0), ("Miandad", 150)],
#     [("Haris", 67), ("Shoaib", 99), ("Babar", 5), ("Faheem", 0), ("Miandad", -1)]
# ]

grid = [ [ ("Saud", 6), ("Babar", 64), ("Rizwan", 3), ("Imam", 24), ("Salman", 42),
("Tayyab", 1), ("Khushdil", 69),
("Shaheen", 14), ("Naseem", 13), ("Haris", 19), ("Abrar", 0) ],
[ ("Imam", 10), ("Babar", 23), ("Saud", 62), ("Rizwan", 46), ("Salman", 19), ("Tayyab",
4), ("Khushdil", 38),
("Shaheen", 0), ("Naseem", 14), ("Haris", 8), ("Abrar", 10) ],
[ ("Babar", 61), ("Rizwan", 43), ("Saud", 55), ("Imam", 14), ("Salman", 39), ("Tayyab",
0), ("Khushdil", -1),
("Shaheen", 7), ("Abrar", -1), ("Naseem", 0), ("Haris", -1)]
]

# Apply sorting
scoreboard = transform_grid_in_place(grid)

# Print the sorted grid
for row in scoreboard:
    print(row)

```

QUESTION 2: BINARY SEARCH OPERATIONS (10 marks)

A sorted list of category groups is given as follows: **categories = [B, D, E, F, H, I, K, M, N, P, R, U]**

Answer the following questions based on the binary search process, by showing all the steps and/or calculations. Only giving final answers will **NOT** be given any credit.

1. List the sequence of indices that the binary search algorithm will examine while searching for student ID = I.
2. Suppose the student mistakenly searches for category Q instead. List the sequence of indices examined and explain how binary search determines that Q is not in the array.
3. List the sequence of indices that the binary search algorithm will examine while searching for category = F.
4. If a new category is added to the list while maintaining sorted order, what is the effect on the binary search process when searching for I, and then K? Which indices would now be visited?
5. If E is deleted, how does the search path change for M and D?
6. What is the maximum number of comparisons binary search makes in a sorted array of size 1000?
7. Given a sorted array of 33 elements, what will be the first middle index chosen by binary search? What if the array had 34 elements instead?

SOLUTION:

1. List the sequence of indices that the binary search algorithm will examine while searching for student ID = I.
 - Find middle index to go to index 5, where this value is being searched at the moment and it is indeed I.
Therefore, the sequence of indices is: **5 → SUCCESS**
2. Suppose the student mistakenly searches for category Q instead. List the sequence of indices examined and explain how binary search determines that Q is not in the array.
 - Go to middle index first, i.e. index 5. The element at index 5 = I and $Q > I$, so search the right side of the array, i.e. from index 6 to index 11
 - Then go to the new middle index, i.e. index 8. The element at index 8 = N and $Q > N$, so search the right side of the array, i.e. from index 9 to index 11
 - Then go to the new middle index, i.e. index 10. The element at index 10 = R and $Q < R$, so search the left side of the array, i.e. index 9 only
 - Then go to the new middle index, i.e. index 9. The element at index 9 = P and $Q > P$, so search the right side of the array. However, the ending condition is met so we stop and deduce Q is not present in the array.
Therefore, the sequence of indices is: **5 → 8 → 10 → 9 → FAILURE**
3. List the sequence of indices that the binary search algorithm will examine while searching for category = F.
 - Go to middle index first, i.e. index 5. The element at index 5 = I and $F < I$, so search the left side of the array, i.e. from index 0 to index 4
 - Then go to the new middle index, i.e. index 2. The element at index 2 = E and $F > E$, so search the right side of the array, i.e. from index 3 to index 4
 - Then go to the new middle index, i.e. index 3. The element at index 3 = F, which is the value that's being searched at the moment.
 - Therefore, the sequence of indices is: **5 → 2 → 3 → SUCCESS**
4. If a new category is added to the list while maintaining sorted order, what is the effect on the binary search process when searching for I, and then K? Which indices would now be visited?
 - You may assume any new category here. I am assuming G here:
 - **categories = [B, D, E, F, G, H, I, K, M, N, P, R, U]**
 - Now search for I:
 - Find middle index to go to index 6, where this value is being searched at the moment and it is indeed I.
 - Therefore, the sequence of indices is: **6 → SUCCESS**
 - No change as I is still the middle element
 - Now search for K:
 - Go to middle index first, i.e. index 6. The element at index 6 = I and $K > I$, so search the right side of the array, i.e. from index 7 to index 12.

- Then go to the new middle index, i.e. index 9. The element at index 9 = N and $K < N$, so search the left side of the array, i.e. from index 7 to index 8
 - Then go to the new middle index, i.e. index 7. The element at index 7 = K, which is the value that's being searched at the moment
 - Therefore, the sequence of indices is: **6 → 9 → 7 → SUCCESS**
 - Similar process but for different set of indices instead
5. If E is deleted, how does the search path change for M and D?
- When E is deleted, the updated array will be as follows:
 - **categories = [B, D, F, G, H, I, K, M, N, P, R, U]**
 - Now search for M:
 - Go to middle index first, i.e. index 5. The element at index 5 = I and $M > I$, so search the right side of the array, i.e. from index 6 to index 11.
 - Then go to the new middle index, i.e. index 8. The element at index 8 = N and $M < N$, so search the left side of the array, i.e. from index 6 to index 7
 - Then go to the new middle index, i.e. index 6. The element at index 6 = K, and $M > K$, so search the right side of the array, i.e. index 7.
 - Then go to the new middle index, i.e. index 7. The element at index 7 = M, which is the value that's being searched at the moment
 - Therefore, the sequence of indices is: **5 → 8 → 6 → 7 → SUCCESS**
 - Similar process but for different set of indices instead
 - Now search for D:
 - Go to middle index first, i.e. index 5. The element at index 5 = I and $D < I$, so search the left side of the array, i.e. from index 0 to index 4.
 - Then go to the new middle index, i.e. index 2. The element at index 2 = F and $D < F$, so search the left side of the array, i.e. from index 0 to index 1.
 - Then go to the new middle index, i.e. index 0. The element at index 0 = B and $D > B$, so search the right side of the array, i.e. index 1.
 - Then go to the new middle index, i.e. index 1. The element at index 1 = D, which is the value that's being searched at the moment
 - Therefore, the sequence of indices is: **5 → 2 → 0 → 1 → SUCCESS**
 - Similar process but for different set of indices instead
6. What is the maximum number of comparisons binary search makes in a sorted array of size 1000?
- Search Time in Binary Search by default contains $O(\log n)$. If $n = 1000$, then number of comparisons will be: $\log(1000) = 9.966$
7. Given a sorted array of 33 elements, what will be the first middle index chosen by binary search? What if the array had 34 elements instead?
- For 33 elements: $\text{middle} = (0 + 32)/2 = 16$ hence it will be: **16**
 - For 34 elements: $\text{middle} = (0 + 33)/2 = 16$ hence it will also be: **16**

QUESTION 3: CIRCULAR QUEUE OPERATIONS (10 marks)

A circular queue can be implemented using fixed-sized arrays as well as dictionaries. Assume the given circular queue is implemented using fixed-sized arrays. For a given Circular Queue, called CQ with size = 3 and empty slots represented as None, complete the following table:

OPERATION	RETURN VALUE	CQ CONTENTS	FRONT INDEX	FRONT ELEMENT	REAR INDEX	REAR ELEMENT	TIME COMPLEXITY
CQ.is_empty()	True	[None, None, None]	-	-	-	-	O(1)
CQ.enqueue(33)	-	[33, None, None]	0	33	0	33	O(1)
CQ.enqueue(44)	-	[33, 44, None]	0	33	1	44	O(1)
CQ.enqueue(55)	-	[33, 44, 55]	0	33	2	55	O(1)
CQ.enqueue(66)	is Full/ Error	"	0	33	2	55	O(1)
CQ.dequeue()	33	[None, 44, 55]	1	44	2	55	O(1)
CQ.dequeue()	44	[None, None, 55]	2	55	2	55	O(1)
CQ.enqueue(66)	-	[66, None, 55]	2	55	0	66	O(1)
CQ.is_full()	False	"	2	55	0	66	O(1)
CQ.first()	55	"	2	55	0	66	O(1)